wanted to create and save an order without creating a new local variable.

rails7/e1/ar/new_examples.rb

```
Order.new do |o|
  o.name     = "Dave Thomas"
  # . . .
  o.save
end
```

Finally, Active Record constructors accept a hash of attribute values as an optional parameter. Each entry in this hash corresponds to the name and value of an attribute to be set. This is useful for doing things like storing values from HTML forms into database rows.

rails7/e1/ar/new_examples.rb

```
an_order = Order.new(
  name:     "Dave Thomas",
  email:    "dave@example.com",
  address:  "123 Main St",
  pay_type: "check")
an_order.save
```

Note that in all of these examples we didn't set the `id` attribute of the new row. Because we used the Active Record default of an integer column for the primary key, Active Record automatically creates a unique value and sets the `id` attribute as the row is saved. We can subsequently find this value by querying the attribute:

rails7/e1/ar/new_examples.rb

```
an_order = Order.new
an_order.name = "Dave Thomas"
# ...
an_order.save
puts "The ID of this order is #{an_order.id}"
```

The `new` constructor creates a new `Order` object in memory; we have to remember to save it to the database at some point. Active Record has a convenience method, `create`, that both instantiates the model object and stores it into the